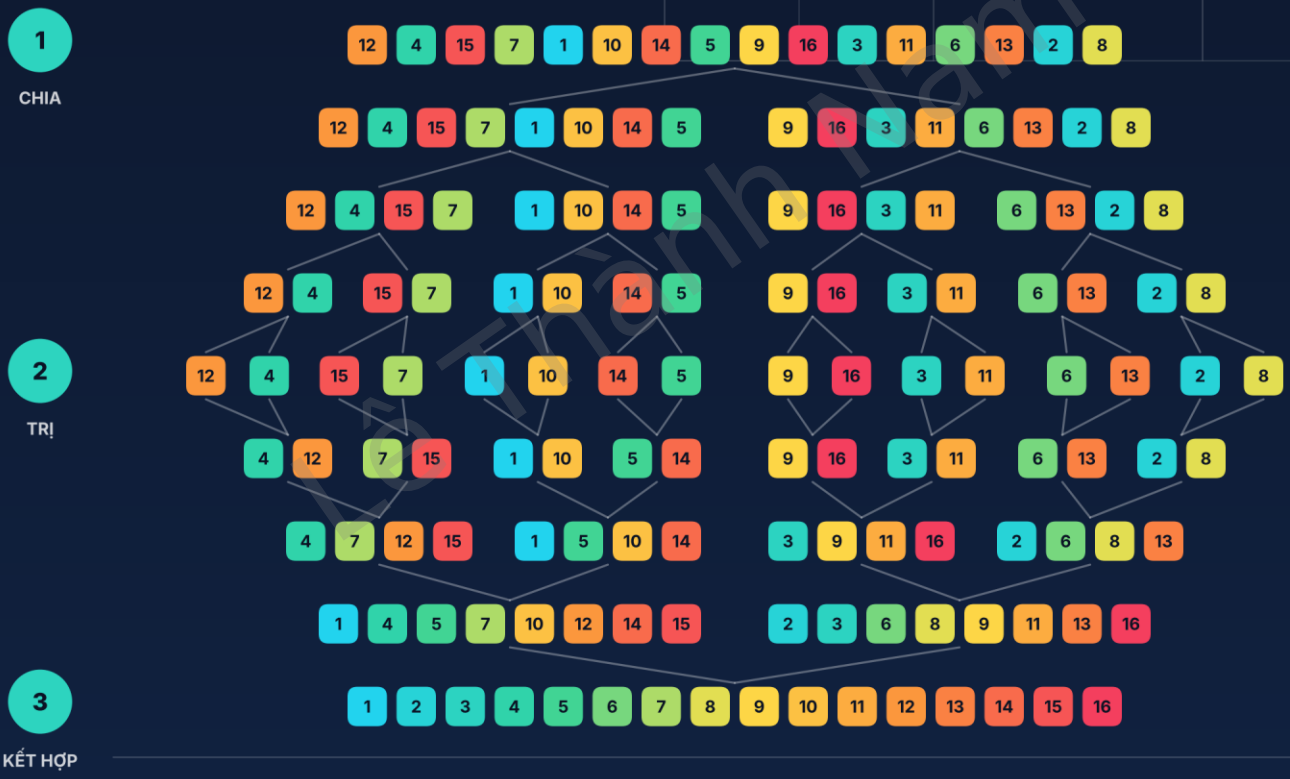
TÀI LIỆU BỒI DƯỠNG HỌC SINH GIỎI TIN HỌC

CHIA ĐỂ TRỊ

Từ cơ bản đến nâng cao

DIVIDE · CONQUER · COMBINE

TẬP 1 · LÝ THUYẾT



Lê Thành Nam

01 Chương 1. NỀN TẢNG CHIA ĐỂ TRỊ

02 Chương 2. TÌM KIẾM NHỊ PHÂN

03 Chương 3. CHIA ĐỂ TRỊ TRÊN DÃY, MẶT PHẲNG VÀ TẬP HỢP

04 Chương 4. CHIA ĐỂ TRỊ NÂNG CAO

14 bài học · 4 chương · từ lũy thừa nhanh đến phân rã trọng tâm

Dành cho học sinh THPT ôn luyện học sinh giỏi cấp tỉnh và tiếp cận cấp quốc gia

Tổng hợp và biên soạn: Lê Thành Nam

LỜI NÓI ĐẦU

Có một câu chuyện quen thuộc về người cha già bảo các con bẻ một bó đũa. Không ai bẻ nổi cả bó, nhưng rút từng chiếc ra thì bẻ dễ như không. Trong Tin học, câu chuyện ấy có một cái tên: chia để trị. Gặp một bài toán quá lớn, ta chia nó thành những bài toán nhỏ hơn cùng dạng, giải từng bài nhỏ (thường bằng chính cách đó, tức là đệ quy), rồi ghép các lời giải nhỏ lại thành lời giải của bài lớn.

Chia để trị là nền móng của rất nhiều thuật toán mà em sẽ dùng suốt đời học Tin: lũy thừa nhanh, sắp xếp trộn, sắp xếp nhanh, tìm kiếm nhị phân, cặp điểm gần nhất, cây phân đoạn... Trong đề thi học sinh giỏi, chia để trị xuất hiện ở hai dạng. Dạng thứ nhất là công cụ: tìm kiếm nhị phân trên đáp án hay đếm nghịch thế bằng sắp xếp trộn là những "viên gạch" có mặt trong rất nhiều lời giải. Dạng thứ hai là ý tưởng chính của câu phân loại: một bài toán tưởng như phải duyệt $O(n^2)$ cặp lại giải được trong $O(n \log n)$, chỉ nhờ một câu hỏi: "Lời giải tốt nhất nằm hẳn bên trái, hẳn bên phải, hay vắt ngang qua điểm giữa?".

Bộ tài liệu này được viết để giúp em nắm vững cả hai dạng ấy một cách có hệ thống, từ lũy thừa nhanh cho tới phân rã trọng tâm cây ở trình độ tiệm cận học sinh giỏi quốc gia. Bộ sách gồm ba tập đi cùng nhau:

- **Tập 1: Lý thuyết** (cuốn sách này) gồm 14 bài học chia thành 4 chương: Nền tảng chia để trị; Tìm kiếm nhị phân; Chia để trị trên dãy, mặt phẳng và tập hợp; Chia để trị nâng cao.
- **Tập 2: Bài tập** gồm 42 bài, mỗi bài học có ba bài tập ở ba mức độ (cơ bản, trung bình, nâng cao), có ràng buộc theo subtask như đề thi thật.
- **Tập 3: Hướng dẫn giải** phân tích từng bài tập từ cách ngây thơ đến cách tối ưu, kèm chương trình hoàn chỉnh và bảng điểm thực tế của từng cách làm.

Mỗi bài học không đưa ra thuật toán ngay. Em sẽ được đặt trước một bài toán cụ thể, thử cách ngây thơ, đếm số phép tính để thấy chỗ bế tắc, rồi được dẫn từng bước để phá bế tắc. Mọi ví dụ đều được chạy tay trên hình vẽ; mọi chương trình trong sách đều đã được kiểm tra bằng cách đối chiếu với lời giải vét cạn trên hàng trăm bộ dữ liệu ngẫu nhiên.

Chương trình được viết bằng C++ theo chuẩn C++14, đọc và ghi tệp theo kiểu "có tệp thì đọc tệp" để dùng được cả với phần mềm chấm Themis lẫn các trang chấm trực tuyến. Tên biến được viết đầy đủ để em đọc code như đọc một đoạn văn.

Chúc các em học tập say mê và đạt kết quả cao!

Lê Thành Nam

Lê Thành Nam

CÁCH HỌC VỚI TÀI LIỆU NÀY

Mỗi bài học đi theo cùng một con đường, giống như cách một người giỏi thật sự suy nghĩ khi gặp bài mới:

Phần	Nội dung
Khởi động	Một câu hỏi nhỏ, gần gũi để em bắt đầu nghĩ.
Bài toán	Đề bài cụ thể, có ví dụ và hình minh họa.
Ý tưởng ngây thơ	Cách làm đầu tiên ai cũng nghĩ ra, kèm sơ đồ khối và phần đếm số phép tính.
Bế tắc	Chỉ ra chính xác vì sao cách ngây thơ chậm, và đặt câu hỏi dẫn tới ý tưởng mới.
Phá bế tắc	Từng bước: quan sát, cách chia, cách kết hợp, chạy tay, chứng minh, chương trình.
Mở rộng	Các biến thể thường gặp trong đề thi.
Bẫy thường gặp	Những lỗi đã làm mất điểm của nhiều thế hệ học sinh.
Ghi nhớ	Tóm tắt những điều cốt lõi nhất.
Luyện tập	Liên hệ tới các bài tập ở Tập 2 và vài câu tự kiểm tra có đáp số.

Các khung màu trong sách có ý nghĩa cố định:

Khung	Ý nghĩa
KHỞI ĐỘNG	Câu hỏi mở đầu bài học.
BÀI TOÁN	Đề bài chính thức.
Ý TƯỞNG CHIA ĐỂ TRỊ	Cách chia, điểm dừng và cách kết hợp.
ĐẾM SỐ PHÉP TÍNH	Phân tích độ phức tạp, giải thích vì sao là $O(n)$, $O(n \log n)$, $O(n^2)$...
BẾ TẮC	Chỗ cách làm hiện tại không vượt qua được.
VÌ SAO ĐÚNG?	Lập luận chứng minh tính đúng đắn.
BẪY THƯỜNG GẶP	Các lỗi hay gặp.
GHI NHỚ	Tóm tắt cuối bài.

Trong các hình vẽ, màu sắc cũng có ý nghĩa cố định: ô **vàng** là ô hoặc đoạn đang xét; ô **xanh lá** là phần mà bước hiện tại dựa vào; ô **xanh dương** là phần đã xử lí xong; ô **trắng** là phần chưa xét; ô **tím** là trường hợp cơ sở; ô **đỏ** là phần bị loại hoặc cách làm sai. Khi một dãy bị chia đôi, nửa trái tô **xanh ngọc** và nửa phải tô **hồng**.

Lời khuyên khi học: Hãy đọc với giấy và bút bên cạnh. Mỗi khi gặp một hình chia đôi, hãy tự vẽ lại cây đệ quy trước khi xem hình. Mỗi khi gặp một thuật toán chia để trị, hãy tự trả lời bốn câu hỏi: **chia** thành những bài con nào, **điểm dừng** ở đâu, **kết hợp** các lời giải con ra sao, và **chi phí** của cả cây đệ quy là bao nhiêu. Chỉ khi tự viết lại được chương trình mà không nhìn sách, em mới thật sự nắm được bài.

Lê Thành Nam

MỤC LỤC

LỜI NÓI ĐẦU **2**

CÁCH HỌC VỚI TÀI LIỆU NÀY **3**

CHƯƠNG 1. NỀN TẢNG CHIA ĐỂ TRỊ **6**

Bài 1. Chia, trị, kết hợp: Lũy thừa nhanh 6
Bài 2. Sắp xếp trộn và đếm nghịch thế 20
Bài 3. Sắp xếp nhanh, phần tử thứ k và cây đệ quy 34

CHƯƠNG 2. TÌM KIẾM NHỊ PHÂN **45**

Bài 4. Tìm kiếm nhị phân trên dãy đã sắp xếp 45
Bài 5. Tìm kiếm nhị phân trên đáp án 56
Bài 6. Tìm kiếm nhị phân trên số thực và tìm kiếm tam phân 68

CHƯƠNG 3. CHIA ĐỂ TRỊ TRÊN DÃY, MẶT PHẲNG VÀ TẬP HỢP **79**

Bài 7. Lời giải đi qua điểm giữa 79
Bài 8. Cặp điểm gần nhất 91
Bài 9. Gặp nhau ở giữa 99
Bài 10. Cấu trúc tự đồng dạng 109

CHƯƠNG 4. CHIA ĐỂ TRỊ NÂNG CAO **118**

Bài 11. Cây phân đoạn: Chia để trị được lưu lại 118
Bài 12. Tối ưu quy hoạch động bằng chia để trị 127
Bài 13. Chia để trị theo thời gian (CDQ) 135
Bài 14. Phân rã trọng tâm cây 142

PHỤ LỤC: BẢNG TRA CỨU NHANH **150**

Lê Thành Nam

CHƯƠNG 1

NỀN TẢNG CHIA ĐỂ TRỊ

*Muốn bẻ một bó đũa, hãy bẻ từng chiếc. Muốn giải một bài toán lớn, hãy chia nó thành những bài toán nhỏ hơn cùng dạng, giải từng bài nhỏ rồi ghép lại. Chương này xây nền móng cho cả cuốn sách: Em sẽ biến một phép tính cần ba mươi năm thành một phép tính chưa tới một phần triệu giây, tự tay sắp xếp một triệu số bằng cách "chia đôi rồi trộn", và học cách nhìn một thuật toán đệ quy dưới dạng một **cây** để đếm chính xác nó tốn bao nhiêu phép tính.*

■ **TRONG CHƯƠNG NÀY**

- **Bài 1.** Chia, trị, kết hợp: Lũy thừa nhanh và tổng cấp số nhân
- **Bài 2.** Sắp xếp trộn và đếm nghịch thế
- **Bài 3.** Sắp xếp nhanh, phần tử thứ k và cây đệ quy

BÀI 1 Chia, trị, kết hợp: Lũy thừa nhanh

Học xong bài này, em sẽ:

✓ Thấy tận mắt vì sao "nhân lần lượt" là vô vọng khi số mũ lên tới 10^{18}, còn "chia đôi số mũ" chỉ cần khoảng 120 phép nhân.

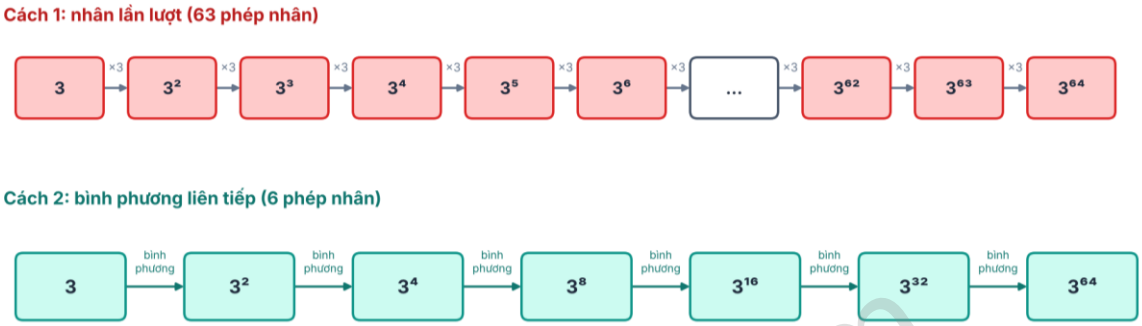
✓ Nắm được **ba bước** của mọi thuật toán chia để trị: **Chia**, **Trị**, **Kết hợp**, cùng với **điểm dừng**.

✓ Hiểu vì sao phải gọi đệ quy **một lần** rồi dùng kết quả hai lần, và cách đếm số phép tính bằng cây đệ quy.

✓ Áp dụng cùng ý tưởng cho bài **tổng cấp số nhân** theo modulo bất kì, nơi công thức quen thuộc thất bại.

■ KHỞI ĐỘNG

Hãy tính 3^{64} bằng tay. Cách tự nhiên nhất là lấy 3 nhân 3, được 9, lại nhân 3, được 27, ... và cứ thế **63 lần**. Nhưng hãy thử cách khác: 3 bình phương được 3^2, bình phương tiếp được 3^4, rồi 3^8, 3^{16}, 3^{32}, 3^{64}. Chỉ **6 phép nhân**! Vậy còn 3^{13} thì sao, khi 13 không phải là lũy thừa của 2? Và nếu số mũ là một tỉ tỉ (10^{18}) thì cần bao nhiêu phép nhân? Cuối bài này, em sẽ thấy câu trả lời là chưa tới **120**.



Hình 1.1. *Tính 3^{64}: 63 phép nhân (hàng trên) hay chỉ 6 phép bình phương (hàng dưới)*

■ BÀI TOÁN: LUYTHUA

Cho ba số nguyên a, b, m. Hãy tính $a^b \bmod m$, tức là số dư khi chia a^b cho m. **Dữ liệu vào:** Một dòng gồm ba số nguyên a, b, m ($0 \le a \le 10^9$, $0 \le b \le 10^{18}$, $1 \le m \le 10^9$). **Kết quả:** Một số nguyên là $a^b \bmod m$.

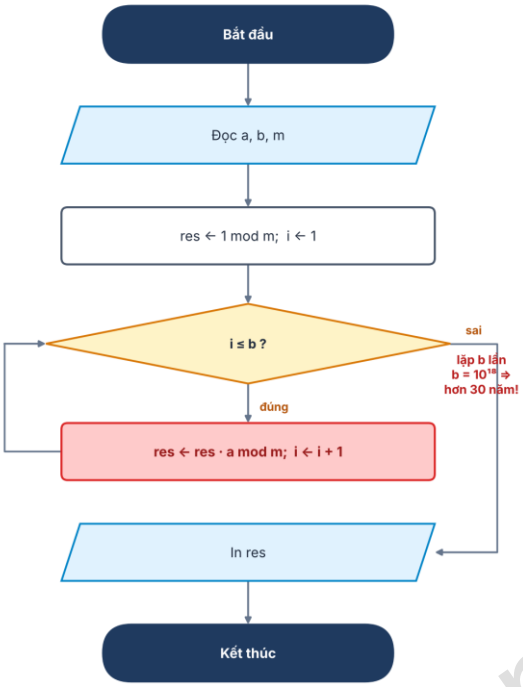
Dữ liệu vào	Kết quả	Giải thích
3 13 1000	323	$3^{13} = 1\,594\,323$, ba chữ số cuối là 323.
2 10 1000	24	$2^{10} = 1024$.
5 0 1	0	$5^0 = 1$, nhưng 1 chia cho 1 dư 0.

Ví dụ thứ ba nhắc ta một chi tiết dễ quên: Khi m = 1 thì **mọi** số chia cho m đều dư 0, kể cả $a^0 = 1$.

1. Ý tưởng ngây thơ: Nhân lần lượt

Cách đầu tiên ai cũng nghĩ ra: Bắt đầu từ kết quả bằng 1, nhân với a đúng b lần. Để số không bị tràn, ta lấy dư cho m **sau mỗi phép nhân**. Điều này hợp lệ vì phép lấy dư "đi xuyên qua" phép nhân: $(x \cdot y) \bmod m = ((x \bmod m) \cdot (y \bmod m)) \bmod m$.

Lê Thành Nam



Hình 1.2. Sơ đồ khối của cách nhân lần lượt. Ô đỏ được lặp lại đúng b lần

‹› Code ý chính: Nhân lần lượt, O(b)

```
long long res = 1 % mod;
for (long long step = 1; step <= b; step++)
    res = res * a % mod;
```

Vì a và res đều nhỏ hơn $m \le 10^9$ (sau khi đã lấy dư a cho m), tích của chúng nhỏ hơn 10^{18}, vừa kiểu long long (giới hạn khoảng $9{,}2 \cdot 10^{18}$).

■ ĐẾM SỐ PHÉP TÍNH

- Vòng lặp chạy đúng b lần, mỗi lần một phép nhân và một phép lấy dư. Độ phức tạp là **O(b)**.
- Một máy tính chấm bài thực hiện khoảng 10^8 đến 10^9 phép tính đơn giản mỗi giây. Với $b = 10^6$: một phần nghìn giây. Với $b = 10^9$: khoảng 3 giây, đã quá thời gian.
- Với $b = 10^{18}$: khoảng **$3 \cdot 10^9$ giây**, tức là gần **một trăm năm**.

■ BẾ TẮC

Mỗi phép nhân chỉ làm số mũ của kết quả tăng thêm **1**. Muốn đi từ số mũ 1 tới số mũ 10^{18} theo từng bước +1 thì không có cách nào nhanh hơn 10^{18} bước. Phải có cách làm số mũ tăng **nhanh hơn**: chẳng hạn **gấp đôi** sau mỗi bước, như phép bình phương ở phần Khởi động.

Lê Thành Nam

2. Phá bế tắc: Chia đôi số mũ

Bước 1 Quan sát: a^b được ghép từ hai nửa giống hệt nhau

Với b chẵn, chẳng hạn b = 12, ta tách mười hai thừa số a thành hai nhóm, mỗi nhóm sáu thừa số:

$$a^{12} = (a \cdot a \cdot a \cdot a \cdot a \cdot a) \cdot (a \cdot a \cdot a \cdot a \cdot a \cdot a) = a^6 \cdot a^6$$

Với b lẻ, chẳng hạn b = 13, còn thừa đúng một thừa số a: $a^{13} = a^6 \cdot a^6 \cdot a$. Tổng quát, gọi $h = a^{\lfloor b/2 \rfloor}$ ($\lfloor b/2 \rfloor$ là phần nguyên của b/2, trong C++ chính là b / 2):

> ■ **CÔNG THỨC CHIA ĐÔI SỐ MŨ**
>
> $\mathbf{a^b = h \cdot h}$ nếu b chẵn; $\mathbf{a^b = h \cdot h \cdot a}$ nếu b lẻ; với $\mathbf{h = a^{\lfloor b/2 \rfloor}}$. Điểm dừng: $\mathbf{a^0 = 1}$.

Đây là một quan sát rất mạnh: Bài toán "tính a^b" được đưa về bài toán **cùng dạng** "tính $a^{\lfloor b/2 \rfloor}$", với số mũ chỉ còn **một nửa**. Bài toán nhỏ hơn đó lại được đưa về số mũ một phần tư, một phần tám, ... cho tới khi số mũ bằng 0.

Bước 2 Viết thành hàm đệ quy

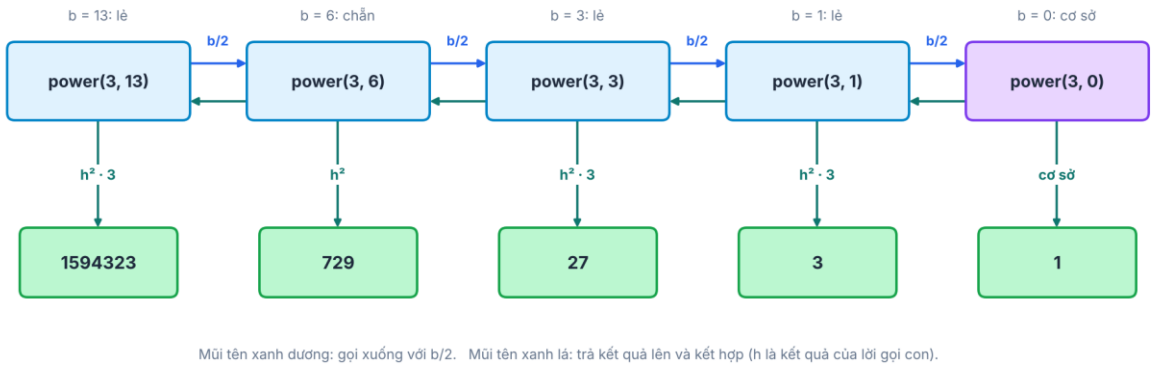
Công thức trên có thể chép gần như nguyên văn thành chương trình. Hàm power(a, b) tự gọi chính nó với số mũ b / 2, nhận về h, rồi bình phương h (và nhân thêm a nếu b lẻ):

‹› Code ý chính: Lũy thừa nhanh bằng đệ quy

```
long long power(long long a, long long b) {
    if (b == 0) return 1 % mod;          // điểm dừng
    long long half = power(a, b / 2);    // TRỊ bài toán con (chỉ một lần!)
    long long res = half * half % mod;   // KẾT HỢP
    if (b % 2 == 1) res = res * a % mod;
    return res;
}
```

Bước 3 Chạy tay với a = 3, b = 13, m = 1000

Lời gọi power(3, 13) gọi power(3, 6), lời gọi này gọi power(3, 3), rồi power(3, 1), rồi power(3, 0). Tới đây gặp điểm dừng, các kết quả lần lượt được **trả ngược lên** và kết hợp:



Hình 1.3. Chuỗi lời gọi của power(3, 13). Đi xuống: chia đôi số mũ. Đi lên: bình phương, nhân thêm 3 khi số mũ lẻ (giá trị trong hình chưa lấy dư)

Lê Thành Nam

Bảng dưới ghi lại từng lời gọi, **từ dưới lên** theo đúng thứ tự các kết quả được tính xong. Khi tự làm, em hãy kẻ đúng một bảng như thế này: mỗi dòng ghi số mũ b, giá trị h nhận từ lời gọi con, và phép kết hợp cụ thể.

Lời gọi	b chẵn hay lẻ	h = power(3, b/2)	Phép kết hợp (lấy dư 1000)	Kết quả
power(3, 0)	điểm dừng	không gọi	1	**1**
power(3, 1)	lẻ	1	$1 \cdot 1 \cdot 3 = 3$	**3**
power(3, 3)	lẻ	3	$3 \cdot 3 \cdot 3 = 27$	**27**
power(3, 6)	chẵn	27	$27 \cdot 27 = 729$	**729**
power(3, 13)	lẻ	729	$729 \cdot 729 = 531\,441 \to 441$; $441 \cdot 3 = 1\,323 \to 323$	**323**

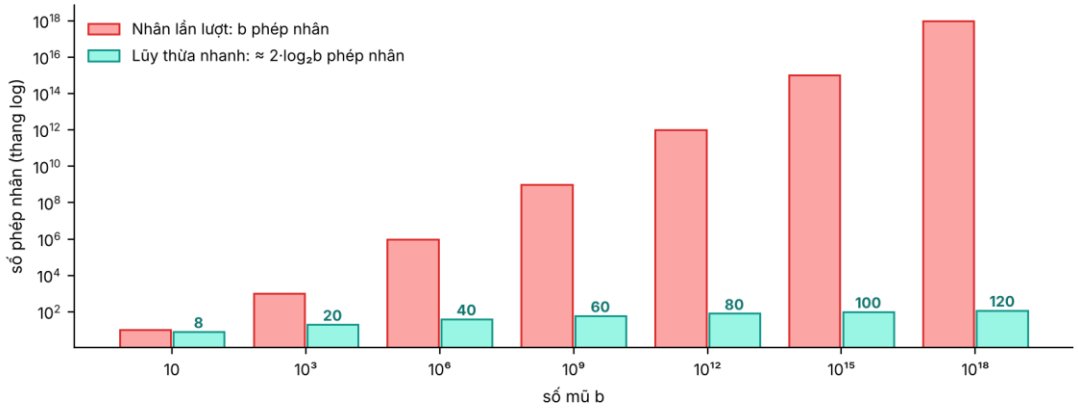
Kết quả 323 khớp với ví dụ thứ nhất. Chỉ cần **5 lời gọi** và **8 phép nhân**, thay vì 13 phép nhân của cách ngây thơ. Với số mũ nhỏ như 13, khoản tiết kiệm còn khiêm tốn; nhưng hãy xem điều gì xảy ra với số mũ lớn.

Bước 4 Đếm số phép tính

Mỗi lời gọi làm số mũ giảm **ít nhất một nửa**: từ b xuống $\lfloor b/2 \rfloor$. Bắt đầu từ b, sau một lời gọi còn không quá b/2, sau hai lời gọi còn không quá b/4, ..., sau k lời gọi còn không quá $b/2^k$. Số mũ chạm 0 khi $2^k > b$, tức là sau khoảng $\log_2 b + 1$ lời gọi.

▪ ĐẾM SỐ PHÉP TÍNH

- Gọi T(b) là số phép tính để tính power(a, b). Mỗi lời gọi tốn một lượng việc cố định (tối đa hai phép nhân, hai phép lấy dư) cộng với lời gọi con: **T(b) = T(b/2) + O(1)**.
- Trải ra: $T(b) = T(b/2) + c = T(b/4) + 2c = \ldots = T(1) + c \cdot \log_2 b$. Vậy độ phức tạp là **O(log b)**.
- Với $b = 10^{18}$: $\log_2 10^{18} \approx 59{,}8$, nên có **60 lời gọi** và không quá **120 phép nhân**. Từ gần một trăm năm xuống dưới một phần triệu giây.



Hình 1.4. Số phép nhân của hai cách theo số mũ b (thang logarit): cột đỏ tăng theo b, cột xanh chỉ tăng thêm khoảng 20 khi b gấp nghìn lần

Lê Thành Nam

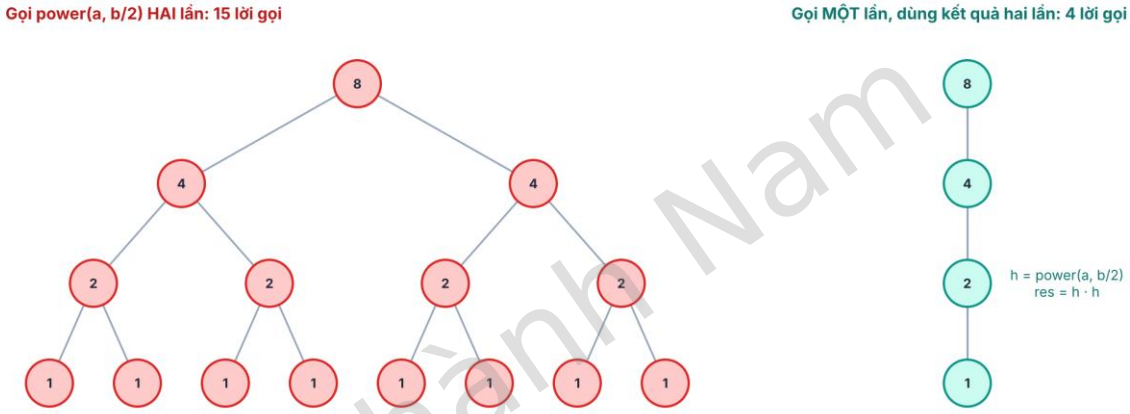
Bước 5 Một lỗi nhỏ phá hỏng tất cả: gọi hai lần

Nhiều bạn viết công thức "$a^b = a^{b/2} \cdot a^{b/2}$" thành chương trình **nguyên văn**:

‹› Code ý chính: SAI về độ phức tạp, gọi đệ quy hai lần

```
long long power(long long a, long long b) {
    if (b == 0) return 1 % mod;
    long long res = power(a, b / 2) * power(a, b / 2) % mod;   // gọi HAI lần!
    if (b % 2 == 1) res = res * a % mod;
    return res;
}
```

Chương trình này cho kết quả đúng, nhưng chậm y như cách ngây thơ! Hãy vẽ các lời gọi với b = 8: power(8) gọi hai lần power(4); mỗi power(4) gọi hai lần power(2); mỗi power(2) gọi hai lần power(1). Tổng cộng 1 + 2 + 4 + 8 = **15 lời gọi**, và số lời gọi luôn vào khoảng **2b**.



Hình 1.5. Bên trái: gọi hai lần, cây lời gọi nhân đôi ở mỗi tầng. Bên phải: gọi một lần, chỉ là một chuỗi

Lê Thành Nam

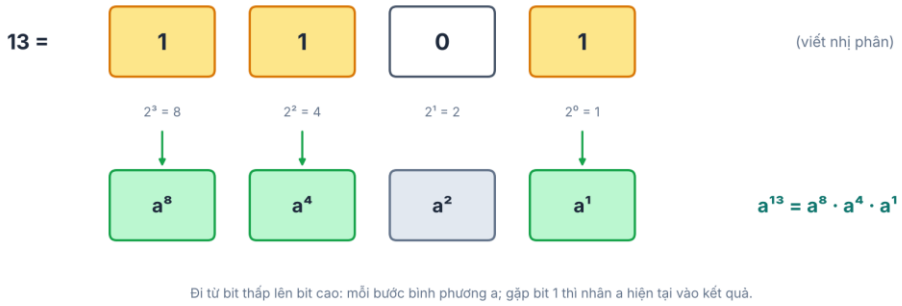
■ SO SÁNH HAI CÔNG THỨC TRUY HỒI

- Gọi một lần: **T(b) = T(b/2) + O(1)**. Cây lời gọi là một **chuỗi** dài $\log_2 b$ nút: **O(log b)**.
- Gọi hai lần: **T(b) = 2·T(b/2) + O(1)**. Cây lời gọi là một **cây nhị phân đầy đủ**: tầng 0 có 1 nút, tầng 1 có 2 nút, tầng 2 có 4 nút, ..., tầng cuối có khoảng b nút. Tổng cộng khoảng 2b nút: **O(b)**.
- Hai lời gọi power(a, b/2) tính ra **cùng một giá trị**. Tính một lần rồi dùng hai lần là đủ.

Đây là bài học quan trọng nhất của Bài 1, và em sẽ gặp lại nó nhiều lần: **Độ phức tạp của một thuật toán chia để trị phụ thuộc vào số bài toán con, kích thước của chúng, và chi phí kết hợp.** Chỉ cần đổi "một bài con" thành "hai bài con giống nhau", độ phức tạp đã nhảy từ O(log b) lên O(b).

Bước 6 Cách viết không đệ quy: Đọc số mũ theo bit

Viết số mũ dưới dạng nhị phân: $13 = 1101_2 = 8 + 4 + 1$. Khi đó $a^{13} = a^8 \cdot a^4 \cdot a^1$. Các lũy thừa a^1, a^2, a^4, a^8, ... được tạo ra bằng cách **bình phương liên tiếp**; ta chỉ việc nhân vào kết quả những lũy thừa ứng với **bit 1** của b.



Hình 1.6. *Số mũ 13 viết nhị phân là 1101: chỉ ba lũy thừa a^8, a^4, a^1 được nhân vào kết quả*

‹› Code ý chính: Lũy thừa nhanh không đệ quy

```
long long res = 1 % mod;
a %= mod;
while (b > 0) {
   if (b & 1) res = res * a % mod;   // bit thấp nhất của b là 1
   a = a * a % mod;                  // a^1 -> a^2 -> a^4 -> a^8 ...
   b >>= 1;                          // bỏ bit thấp nhất
}
```

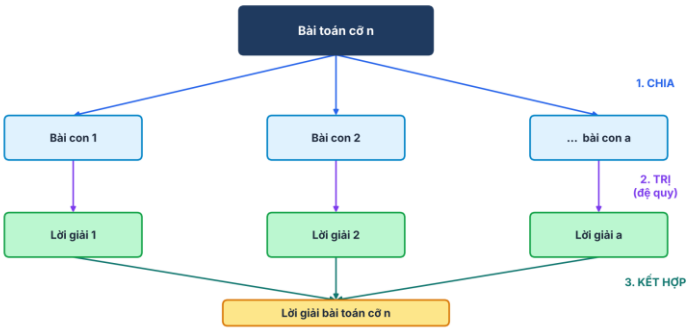
Chạy tay với a = 3, b = 13, m = 1000. Mỗi dòng là một lần lặp; cột "a hiện tại" là lũy thừa 3^1, 3^2, 3^4, 3^8 (đã lấy dư):

Lần lặp	b (nhị phân)	Bit thấp nhất	a hiện tại	res sau lần lặp
1	13 (1101)	1	$3 = 3^1$	$1 \cdot 3 =$ **3**
2	6 (110)	0	$9 = 3^2$	**3** (không nhân)
3	3 (11)	1	$81 = 3^4$	$3 \cdot 81 =$ **243**
4	1 (1)	1	$6561 \to 561 = 3^8 \bmod 1000$	$243 \cdot 561 = 136\,323 \to$ **323**

Hai cách viết có cùng độ phức tạp O(log b). Cách đệ quy gần với ý tưởng "chia để trị" hơn; cách lặp ngắn và không tốn bộ nhớ ngăn xếp. Em nên thuộc cả hai.

3. Chia để trị là gì?

Điều em vừa làm có tên là **chia để trị** (tiếng Anh: divide and conquer). Mọi thuật toán chia để trị đều gồm ba bước, cộng với một điểm dừng:



Hình 1.7. *Ba bước của chia để trị: Chia bài toán thành các bài con, Trị từng bài con bằng đệ quy, Kết hợp các lời giải con*

Lê Thành Nam

■ BA BƯỚC CỦA CHIA ĐỂ TRỊ

1. Chia (divide): Tách bài toán thành một hay nhiều bài toán con **cùng dạng nhưng nhỏ hơn**. Ở bài LUYTHUA: a^b được đưa về $a^{\lfloor b/2 \rfloor}$.

2. Trị (conquer): Giải các bài toán con, thường bằng chính thuật toán đang viết (đệ quy). Khi bài toán đủ nhỏ thì giải trực tiếp: đó là **điểm dừng**. Ở bài LUYTHUA: $a^0 = 1$.

3. Kết hợp (combine): Ghép lời giải các bài con thành lời giải bài lớn. Ở bài LUYTHUA: bình phương h, nhân thêm a nếu b lẻ.

Khi thiết kế một thuật toán chia để trị, hãy tập thói quen viết ra giấy câu trả lời cho **bốn câu hỏi** sau **trước khi** viết dòng code đầu tiên:

■ BỐN CÂU HỎI CỦA MỌI BÀI CHIA ĐỂ TRỊ

1. Chia thế nào? Bài toán con là gì? Phát biểu thành một câu rõ ràng, đầy đủ. Ở đây: "tính $a^{\lfloor b/2 \rfloor}$ mod m".

2. Điểm dừng ở đâu? Bài toán nhỏ nhất nào biết ngay đáp án? Ở đây: b = 0, đáp án 1 mod m.

3. Kết hợp thế nào? Từ lời giải các bài con, làm sao có lời giải bài lớn? Ở đây: h · h (· a nếu b lẻ).

4. Tốn bao nhiêu? Có bao nhiêu bài con, mỗi bài cỡ bao nhiêu, kết hợp tốn bao nhiêu? Từ đó viết công thức truy hồi T(n) và tính độ phức tạp. Ở đây: T(b) = T(b/2) + O(1) = O(log b).

■ CHIA ĐỂ TRỊ VÀ QUY HOẠCH ĐỘNG

Nếu em đã học quy hoạch động, em sẽ thấy hai phương pháp rất giống nhau: cả hai đều chia bài toán thành các bài toán con cùng dạng. Điểm khác biệt nằm ở **các bài con có gối nhau hay không**. Trong quy hoạch động, cùng một bài con xuất hiện ở rất nhiều nhánh, nên ta phải **lưu bảng** để khỏi tính lại. Trong chia để trị điển hình, các bài con **độc lập**, không trùng nhau (chẳng hạn hai nửa rời nhau của một dãy ở Bài 2), nên không cần bảng: chỉ cần đệ quy và kết hợp. Phiên bản "gọi hai lần" ở Bước 5 chính là một trường hợp bài con trùng nhau; cách sửa của ta (tính một lần, dùng hai lần) cũng chính là tư tưởng của quy hoạch động.

4. Bài toán TONGCSN: Khi bước kết hợp cần thêm thông tin

Lũy thừa nhanh có bước kết hợp rất đơn giản. Bài toán sau đây cho thấy một tình huống hay gặp hơn: **lời giải bài con không đủ để kết hợp**, ta phải bắt hàm đệ quy trả về **thêm thông tin**.

Lê Thành Nam

■ BÀI TOÁN: TONGCSN

Cho ba số nguyên a, n, m. Hãy tính $S(n) = 1 + a + a^2 + ... + a^{n-1}$ (tổng n số hạng đầu của cấp số nhân công bội a), lấy dư cho m.

Dữ liệu vào: Một dòng gồm a, n, m ($0 \le a \le 10^9$, $0 \le n \le 10^{18}$, $1 \le m \le 10^9$).

Lưu ý: m không nhất thiết là số nguyên tố. Kết quả: S(n) mod m.

Dữ liệu vào	Kết quả	Giải thích
2 13 1000	191	1 + 2 + 4 + ... + 4096 = 8191.
3 2 4	0	1 + 3 = 4, chia hết cho 4.

Hai cách làm "trông như được"

Cách cộng lần lượt giữ một biến lũy thừa và một biến tổng, cộng dồn n số hạng: O(n), quá chậm với $n = 10^{18}$, cùng bế tắc như cách nhân lần lượt.

Cách dùng công thức. Ở lớp 11, em đã học $S(n) = (a^n - 1) / (a - 1)$. Tử số tính được bằng lũy thừa nhanh. Nhưng **phép chia không "đi xuyên qua" phép lấy dư** như phép cộng và phép nhân. Muốn chia cho (a − 1) theo modulo m, ta phải nhân với **nghịch đảo modulo** của (a − 1), mà nghịch đảo này chỉ tồn tại khi (a − 1) và m nguyên tố cùng nhau. Ví dụ thứ hai: a − 1 = 2 không có nghịch đảo modulo 4, công thức không dùng được. Khi $a \equiv 1 \pmod m$, mẫu số còn bằng 0.

■ BẾ TẮC

Ta cần một cách tính S(n) chỉ dùng **cộng và nhân** (hai phép toán "đi xuyên qua" phép lấy dư), nhưng lại nhanh như lũy thừa nhanh. Liệu có thể chia đôi **số số hạng** như đã chia đôi số mũ?

Phá bế tắc: Nửa sau là nửa trước nhân với a^k

Hãy viết S(12) ra và chia thành hai nửa, mỗi nửa sáu số hạng:

$$S(12) = (1 + a + a^2 + a^3 + a^4 + a^5) + (a^6 + a^7 + a^8 + a^9 + a^{10} + a^{11})$$

Đặt a^6 ra ngoài nửa sau: $a^6 + a^7 + ... + a^{11} = a^6 \cdot (1 + a + ... + a^5) = a^6 \cdot S(6)$. Vậy **hai nửa liên quan chặt chẽ với nhau**: nửa sau chính là nửa trước nhân với a^6. Do đó $S(12) = S(6) + a^6 \cdot S(6) = S(6) \cdot (1 + a^6)$. Với số số hạng lẻ, ví dụ S(13), ta tách riêng số hạng cuối a^{12}:



Hình 1.8. Tách S(13): sáu số hạng đầu (xanh) là S(6), sáu số hạng sau (hồng) là $a^6 \cdot S(6)$, cộng thêm số hạng cuối a^{12} (vàng)

Lê Thành Nam

■ CÔNG THỨC CHIA ĐÔI CHO TỔNG CẤP SỐ NHÂN

Với k = ⌊n/2⌋: **S(2k) = S(k) · (1 + a^k) S(2k + 1) = S(2k) + a^{2k}** Điểm dừng: **S(0) = 0**.

Công thức cần cả S(k) **và** a^k. Ta có thể gọi thêm hàm power để tính a^k, nhưng như vậy mỗi tầng tốn thêm O(log n), tổng cộng O($\log^2$ n). Cách gọn hơn: Bắt hàm đệ quy **trả về một cặp** (S(n), a^n). Khi đã có cặp (S(k), a^k) của bài con, ta tính được cả hai thành phần của bài lớn:

- S(2k) = S(k) · (1 + a^k), $a^{2k} = a^k \cdot a^k$;
- nếu n = 2k + 1 lẻ: S(2k + 1) = S(2k) + a^{2k}, $a^{2k+1} = a^{2k} \cdot a$.

Đây là một kĩ thuật rất phổ biến: **làm mạnh giả thiết quy nạp**. Hàm đệ quy trả về nhiều hơn những gì đề bài hỏi, chính vì phần "nhiều hơn" ấy giúp bước kết hợp trở nên dễ dàng. Em sẽ gặp lại nó ở Bài 7 (đoạn con có tổng lớn nhất) và Bài 11 (cây phân đoạn).

Chạy tay với a = 2, n = 13, m = 1000

Các lời gọi đi xuống theo n = 13 → 6 → 3 → 1 → 0, rồi kết quả được trả ngược lên:

Lời gọi	Cặp của bài con (S(k), a^k)	S(2k) = S(k)(1 + a^k); a^{2k}	Nếu n lẻ: + a^{2k}; · a	Kết quả (S(n), a^n)
n = 0	điểm dừng			**(0, 1)**
n = 1	(0, 1)	0 · 2 = 0; 1	0 + 1 = 1; 1 · 2 = 2	**(1, 2)**
n = 3	(1, 2)	1 · 3 = 3; 4	3 + 4 = 7; 4 · 2 = 8	**(7, 8)**
n = 6	(7, 8)	7 · 9 = 63; 64	n chẵn	**(63, 64)**
n = 13	(63, 64)	63 · 65 = 4095 → 95; 4096 → 96	95 + 96 = 191; 96 · 2 = 192	**(191, 192)**

Kết quả S(13) = 191 khớp với ví dụ (8191 mod 1000 = 191). Thành phần thứ hai 192 = 8192 mod 1000 = 2^{13} mod 1000, cũng đúng.

■ VÌ SAO ĐÚNG?

Ta chứng minh bằng quy nạp theo n rằng hàm trả về đúng cặp (S(n), a^n). Với n = 0: S(0) = 0 (tổng rỗng) và a^0 = 1, đúng. Giả sử hàm đúng với k = ⌊n/2⌋ < n. Khi đó:

- S(2k) = (1 + ... + a^{k-1}) + (a^k + ... + a^{2k-1}) = S(k) + a^k · S(k) = S(k)(1 + a^k).
- S(2k + 1) = S(2k) + a^{2k} (thêm đúng số hạng cuối).

Mọi phép toán chỉ gồm cộng và nhân, nên ta được phép lấy dư sau mỗi phép mà không làm sai kết quả, **với mọi m**, nguyên tố hay không.

5. Chương trình hoàn chỉnh

Dưới đây là chương trình hoàn chỉnh cho hai bài. Phần đọc tệp được viết theo kiểu "có tệp thì đọc tệp": Khi chấm bằng Themis với tệp LUYTHUA.INP, chương trình đọc và ghi tệp; khi nộp lên trang chấm trực tuyến (đọc từ bàn phím), điều kiện fopen sai nên chương trình đọc chuẩn như bình thường.

Lê Thanh Nam

LUYTHUA.CPP: Lũy thừa nhanh, O(log b)

```
1  #include <cstdio>
2  #include <iostream>
3  using namespace std;
4
5  long long mod;
6
7  long long power(long long a, long long b) {
8      if (b == 0) return 1 % mod;
9      long long half = power(a, b / 2);
10     long long res = half * half % mod;
11     if (b % 2 == 1) res = res * a % mod;
12     return res;
13 }
14
15 int main() {
16     ios_base::sync_with_stdio(false);
17     cin.tie(nullptr);
18     if (fopen("LUYTHUA.INP", "r")) {
19         freopen("LUYTHUA.INP", "r", stdin);
20         freopen("LUYTHUA.OUT", "w", stdout);
21     }
22     long long a, b;
23     cin >> a >> b >> mod;
24     a %= mod;
25     cout << power(a, b) << '\n';
26     return 0;
27 }
```

Vài chi tiết đáng chú ý:

- Dòng 24 lấy dư a cho m **ngay sau khi đọc**. Nếu quên, a có thể bằng 10^9 trong khi m nhỏ; tích a · a vẫn vừa long long, nhưng với những bài có a tới 10^{18} thì sẽ tràn số.
- Dòng 8 trả về 1 % mod chứ không phải 1, để ví dụ "5 0 1" cho kết quả 0.
- Hàm power gọi chính nó **đúng một lần** (dòng 9). Độ sâu đệ quy không quá 61 tầng, không có nguy cơ tràn ngăn xếp.

Lê Thành Nam

TONGCSN.CPP: Chia đôi số số hạng, O(log n)

```cpp
#include <cstdio>
#include <iostream>
#include <utility>
using namespace std;

long long mod;

// trả về cặp (S(n), a^n) với S(n) = 1 + a + ... + a^(n-1)
pair<long long, long long> solve(long long a, long long n) {
    if (n == 0) return make_pair(0LL, 1 % mod);
    pair<long long, long long> half = solve(a, n / 2);
    long long s = half.first, p = half.second;
    // S(2k) = S(k) * (1 + a^k),  a^(2k) = a^k * a^k
    long long sum = s * ((1 + p) % mod) % mod;
    long long pw = p * p % mod;
    if (n % 2 == 1) {
        // S(2k+1) = S(2k) + a^(2k)
        sum = (sum + pw) % mod;
        pw = pw * a % mod;
    }
    return make_pair(sum, pw);
}

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(nullptr);
    if (fopen("TONGCSN.INP", "r")) {
        freopen("TONGCSN.INP", "r", stdin);
        freopen("TONGCSN.OUT", "w", stdout);
    }
    long long a, n;
    cin >> a >> n >> mod;
    a %= mod;
    cout << solve(a, n).first << '\n';
    return 0;
}
```

Lê Thành Nam

Ở dòng 14, (1 + p) % mod được lấy dư trước khi nhân với s, nhờ vậy cả hai thừa số đều nhỏ hơn m và tích nhỏ hơn 10^{18}.

6. Mở rộng

Mở rộng 1: Nhân hai số lớn theo modulo

Nếu m lên tới 10^{18}, tích hai số nhỏ hơn m có thể tới 10^{36}, tràn long long. Có hai cách. Cách thứ nhất dùng kiểu __int128 của GCC để tính tích. Cách thứ hai lại là chia để trị, lần này áp dụng cho **chính phép nhân**: $x \cdot y = 2 \cdot (x \cdot \lfloor y/2 \rfloor) + x \cdot (y \bmod 2)$. Phép "nhân" được thay bằng phép **cộng** và phép nhân đôi, mỗi phép đều lấy dư và không tràn số (vì $2 \cdot 10^{18} < 9{,}2 \cdot 10^{18}$). Kĩ thuật này đôi khi được gọi là "nhân Ấn Độ", tốn O(log y) phép cộng.

Mở rộng 2: Lũy thừa của những thứ không phải số

Lời giải lũy thừa nhanh chỉ dùng một tính chất của phép nhân: **tính kết hợp**, $(x \cdot y) \cdot z = x \cdot (y \cdot z)$, nhờ đó $a^b = a^{\lfloor b/2 \rfloor} \cdot a^{\lfloor b/2 \rfloor} (\cdot a)$. Mọi phép toán có tính kết hợp đều "lũy thừa nhanh" được: nhân **ma trận** (để tính số Fibonacci thứ 10^{18} trong O(log n), xem Tập Quy hoạch động), hợp **hoán vị** (áp dụng

một phép xáo bài 10^{18} lần), hay thậm chí phép "nhân" chính là phép ghép hai đoạn trong bài TONGCSN.

Mở rộng 3: Chia theo phần tử lớn nhất: Tháp Hà Nội

Không phải bài chia để trị nào cũng chia đôi. Trò chơi Tháp Hà Nội chuyển n đĩa từ cọc 1 sang cọc 3: ta **chia theo đĩa lớn nhất**. Muốn chuyển đĩa n, trước hết phải chuyển $n-1$ đĩa trên nó sang cọc 2, chuyển đĩa n, rồi chuyển $n-1$ đĩa từ cọc 2 sang cọc 3. Số bước là $T(n) = 2T(n-1) + 1 = 2^n - 1$. Bài tập 1.3 hỏi trạng thái các cọc sau đúng k bước, với k có thể tới 2^{60}: mô phỏng là vô vọng, nhưng chia để trị theo đĩa lớn nhất trả lời trong n bước.

Mở rộng 4: Số mũ cực lớn

Khi m = p là số nguyên tố và a không chia hết cho p, định lí Fermat nhỏ cho $a^{p-1} \equiv 1 \pmod p$, nên có thể thay số mũ b bằng b mod $(p-1)$. Kĩ thuật này dùng khi số mũ được cho dưới dạng một xâu hàng nghìn chữ số, hoặc chính số mũ cũng là một lũy thừa (như a^{bc}). Hãy cẩn thận với điều kiện "a không chia hết cho p".

■ BẪY THƯỜNG GẶP

- **Gọi đệ quy hai lần** power(a, b/2) · power(a, b/2): vẫn đúng nhưng chậm như cách ngây thơ, O(b).
- **Quên lấy dư a cho m trước khi bắt đầu:** tích a · a có thể tràn số khi a lớn.
- **Trả về 1 khi b = 0:** sai khi m = 1, phải trả về `1 % m`.
- **Dùng hàm `pow` của thư viện cmath:** hàm này tính trên số thực, chỉ chính xác khoảng 15 chữ số, cho kết quả vô nghĩa với a^b khổng lồ.
- **Dùng công thức $(a^n - 1)/(a - 1)$ với phép chia thường hoặc nghịch đảo modulo:** sai khi m không nguyên tố hoặc $a \equiv 1 \pmod m$.
- **Viết `b / 2` với b kiểu `int`** khi đề cho b tới 10^{18}: đọc sai dữ liệu ngay từ đầu. Hãy khai báo `long long`.

■ GHI NHỚ

- Chia để trị = **Chia** thành bài con cùng dạng nhỏ hơn + **Trị** bằng đệ quy (có **điểm dừng**) + **Kết hợp**.
- Lũy thừa nhanh: $a^b = h \cdot h$ (· a nếu b lẻ), $h = a^{\lfloor b/2 \rfloor}$. **Gọi một lần, dùng hai lần**: O(log b).
- Độ phức tạp đọc từ công thức truy hồi: $T(n) = T(n/2) + O(1)$ cho $O(\log n)$; $T(n) = 2T(n/2) + O(1)$ cho $O(n)$.
- Khi bước kết hợp thiếu thông tin, hãy cho hàm đệ quy **trả về thêm** (một cặp, một bộ ba...).

Lê Thành Nam

Luyện tập

Bài	Tên	Gợi ý liên hệ
1.1	NHÂN BẢN	Trả lời 10^5 câu hỏi a^b mod m. Lũy thừa nhanh cho từng câu hỏi; cẩn thận với m = 1.
1.2	TỔNG LŨY THỪA	Tổng $a + a^2 + ... + a^n$ với m bất kì. Biến đổi nhẹ công thức của bài TONGCSN.
1.3	THÁP HÀ NỘI	Trạng thái các cọc sau k bước. Chia theo đĩa lớn nhất: k nhỏ hơn hay không nhỏ hơn 2^{n-1}?

Tự kiểm tra nhanh (đáp số ở cuối mỗi câu):

1. Lời gọi power(2, 100) lần lượt đi qua những số mũ nào? Có bao nhiêu lời gọi? *(Đáp số: 100, 50, 25, 12, 6, 3, 1, 0; tám lời gọi.)*

2. Tính 7^{222} mod 10 bằng tay. *(Đáp số: 9. Các lũy thừa của 7 có chữ số cuối lặp theo chu kì 7, 9, 3, 1; 222 chia 4 dư 2.)*

3. Tính S(5) của bài TONGCSN với a = 3, m = 100 bằng cách chạy tay hàm đệ quy. *(Đáp số: 21. Các cặp trả về: n = 1: (1, 3); n = 2: (4, 9); n = 5: (121 → 21, 243 → 43).)*

4. Hàm đệ quy gọi ba lần chính nó với b/2 thì có độ phức tạp bao nhiêu? *(Đáp số: tầng thứ k có 3^k nút, có $\log_2 b$ tầng, tổng cộng khoảng $3^{\log_2 b} = b^{\log_2 3} \approx b^{1,58}$ lời gọi.)*

Lê Thành Nam